Learning PID control

Valavala Charan Teja
Intern assignment | Modern Monk Labs

1. The basic idea

A controller repeatedly compares what we want with what we measure, then changes an input to reduce the difference. The desired value is the **setpoint**. The difference is the **error**. The thing we are controlling is the **plant**.

Think about adjusting a shower. If the water is colder than you want, turn the hot tap up. As it gets close to the right temperature, make smaller changes. If it is warming quickly, stop turning before it gets too hot. Keep checking, because the water pressure can change.

```
error = target - measurement
e(t) = r(t) - y(t)
```

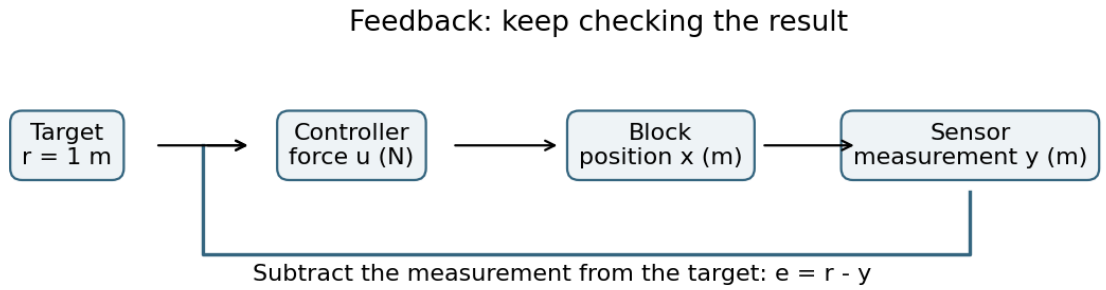


Figure 1. The sensor closes the loop: each new measurement changes the next controller output.

For the block in this assignment, the target is 1 m and the input is force in newtons. If the position is 0.7 m, the error is +0.3 m. A positive force pushes it towards the target. If it moves beyond 1 m, the error becomes negative.

In the response graphs, read time along the bottom and position up the side. The dashed line is the target. A curve crossing above it means overshoot; repeated rises and falls mean oscillation. A step response starts when the target suddenly changes, here from 0 to 1 m.

Feedback helps correct mistakes, but it is not automatically stable. An input that is too strong, too late, or based on a noisy measurement can make the motion worse.

2. The PID formula

PID adds a response to the error now, the error accumulated so far, and how fast the error changes. The integral below adds error over time; de/dt means the change in error per second.

```
u(t) = Kp e(t) + Ki integral[0 to t] e(q) dq
+ Kd de(t)/dt
```

Symbol	Meaning	Unit for this block
t, q	Time; q is the integration variable	s
r, y, e	Target, measured position, error	m
u	Commanded force	N
Kp	Proportional gain	N/m
Ki	Integral gain	N/(m s)
Kd	Derivative gain	N s/m

P: present error. A larger error produces a larger push. P reacts immediately, but its force becomes zero when the error is zero.
I: past error. Small errors add up. This builds the holding force that P cannot supply at zero error.
D: changing error. With a fixed target, D opposes fast motion. It acts like extra damping. Differentiation also magnifies measurement noise.

The small steps used by the code

```
e[n] = r[n] - y[n]
J_trial = J[n-1] + Ki e[n] dt
v[n] = -(y[n] - y[n-1]) / dt
a = tau / (tau + dt)
d[n] = a d[n-1] + (1-a) v[n]
u_raw[n] = Kp e[n] + J[n] + Kd d[n]
u[n] = clip(u_raw[n], lower, upper)
```

n counts updates; dt is the time between them in seconds. J stores the I force in N. v is negative measured velocity, in m/s. d smooths v by mixing its new value with the previous d; a is the weight on the old value. tau sets the smoothing time in seconds. Using -y instead of error avoids a force spike when the target changes. For a fixed target, both give the same derivative. tau = 0 turns smoothing off.

clip keeps force within the limits. Normally J becomes J_trial. Anti-windup rejects increments that push the command further beyond a limit, but allows those that bring it back. Start J and d at zero; the first derivative is zero.

One update with numbers

Use gains (100, 80, 12), dt = tau = 0.02 s, target 1 m, and position 0.80 m after 0.79 m. Previously J = 10 N and d = 0. Error = 0.20 m, so P = 20 N and J_trial = 10 + 80 x 0.20 x 0.02 = 10.32 N. v = -0.50 m/s and a = 0.5, giving d = -0.25 m/s and D = -3 N. Total: **27.32 N**. This is within the 30 N limit, so J_trial is accepted.

3. The test system

The plant is a 1 kg block connected to a spring and a damper. The spring pulls it back towards its rest position. The damper opposes velocity, like a shock absorber.

```
m x'' + b x' + k x = u
m = 1 kg, b = 10 N s/m, k = 20 N/m
x'' = (u - 10 x' - 20 x) / 1
```

x is position in metres, x' is velocity in m/s, and x'' is acceleration in m/s^2. The equation is force balance: applied force must account for acceleration, damping and the spring pull.

At rest, velocity and acceleration are zero. At x = 1 m the spring pulls with 20 N, so the actuator must keep supplying 20 N. The damper needs no force at rest. A 30 N actuator can hold this target, but cannot hold a position above 1.5 m indefinitely.

4. How to judge a result

Rise time: time from first reaching 10% to first reaching 90% of the 1 m target. **Overshoot:** max(0, peak position - 1) / 1, as a percentage. **Settling time:** first sampled time after which the response stays within 0.98 to 1.02 m for the rest of the run. **Final error:** target minus the last position. A finite run only estimates the error after the motion dies away.

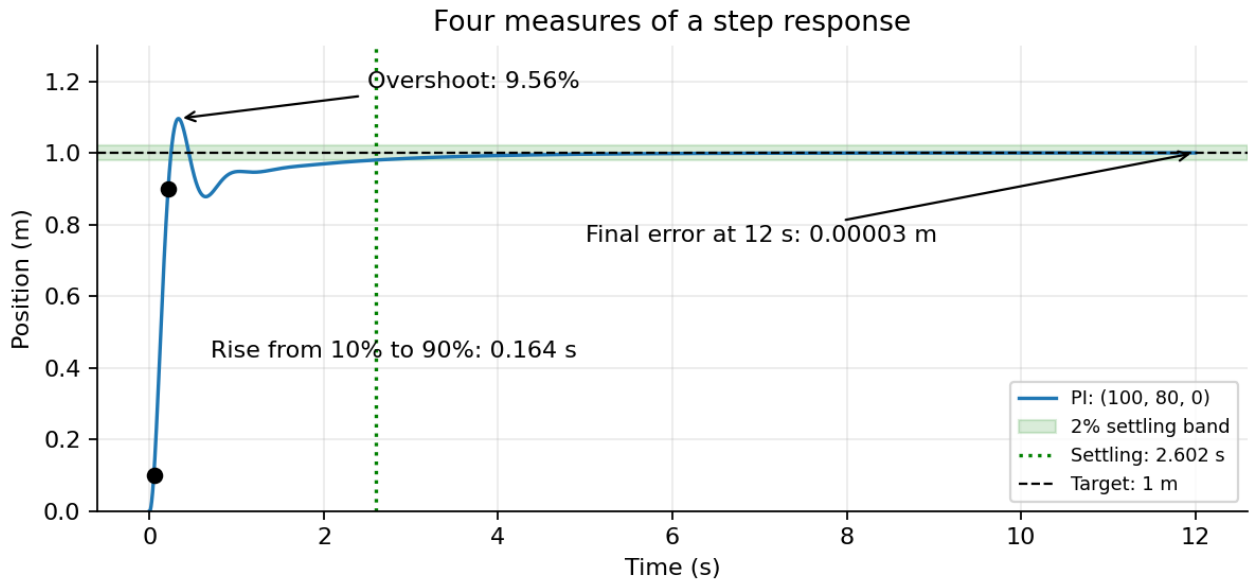


Figure 2. All four measures are marked on the PI response. The two black dots show where the position first reaches 10% and 90% of the target.

The comparison uses 12 s runs, dt = 0.002 s and zero initial position/velocity. Missing rise time means 90% was never reached; missing settling time means the run did not finish inside the band. Rise crossings use linear interpolation. Settling is checked at samples, so it is only resolved to dt.

5. Each part alone

First: P only at four strengths

P applies u = Kp(1 - x). At equilibrium, meaning the block has stopped moving, this force balances the spring. The subscript ss below means steady state: the value after the motion has died away.

```
20 x_ss = Kp (1 - x_ss)
x_ss = Kp / (20 + Kp)
e_ss = 20 / (20 + Kp)
```

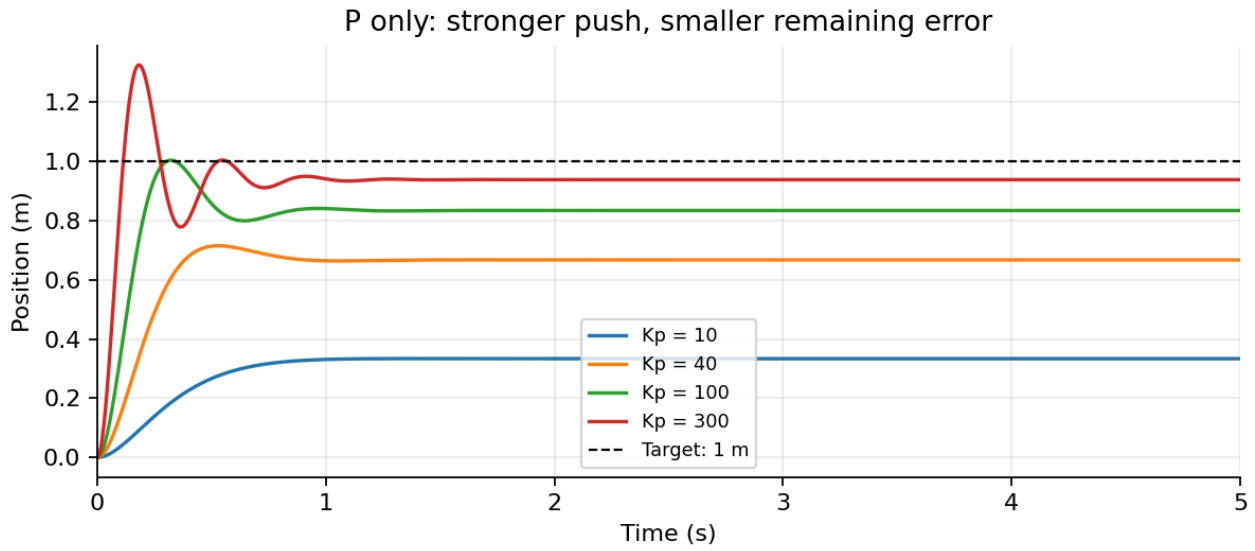


Figure 3. Larger Kp reduces the final error but makes the early response more oscillatory. The curves are simulated for 12 s; the first 5 s are shown.

Kp (N/m)	Predicted x (m)	Simulated x at 12 s (m)	Error (m)
10	0.333333	0.333333	0.666667
40	0.666667	0.666667	0.333333
100	0.833333	0.833333	0.166667
300	0.937500	0.937500	0.062500

The prediction and simulation agree to six decimal places. With finite positive Kp, P alone stops short for this block because a nonzero error is needed to generate a nonzero holding force. That remaining error supplies the force needed to hold the spring.

5. Adding I, then D

Keeping Kp = 100, I adds the missing holding force. Once the position is correct, the integral can retain 20 N even when the error is zero. Increasing Ki too far makes this stored push build faster than the block can respond.

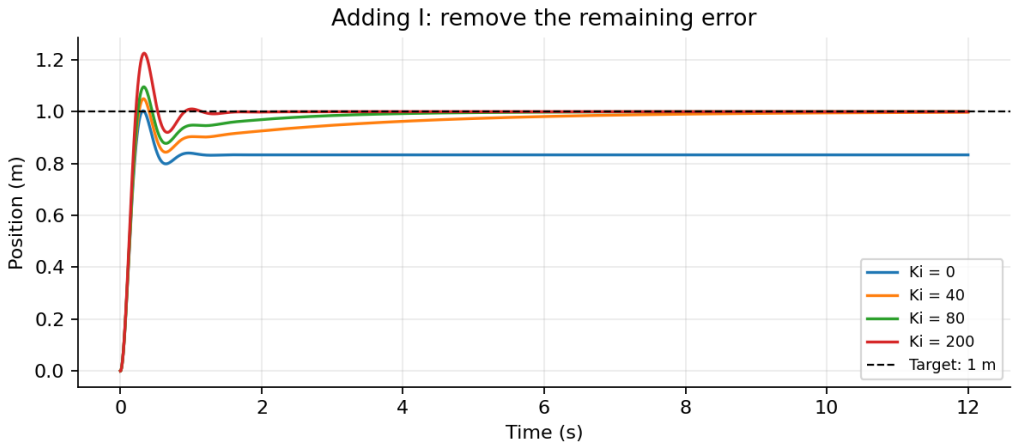


Figure 4. Ki = 40, 80 and 200 remove the offset left by P alone. A larger Ki can increase overshoot.

Next, keep Kp = 100 and Ki = 80 and add D. If the block moves towards the target quickly, D reduces the push before it arrives. To see the damping effect in the equation, temporarily leave I out: ideal PD gives x'' + (10 + Kd)x' + (20 + Kp)x = Kp r. Kd adds to the coefficient of velocity, just like extra damping. D becomes zero when the block stops.

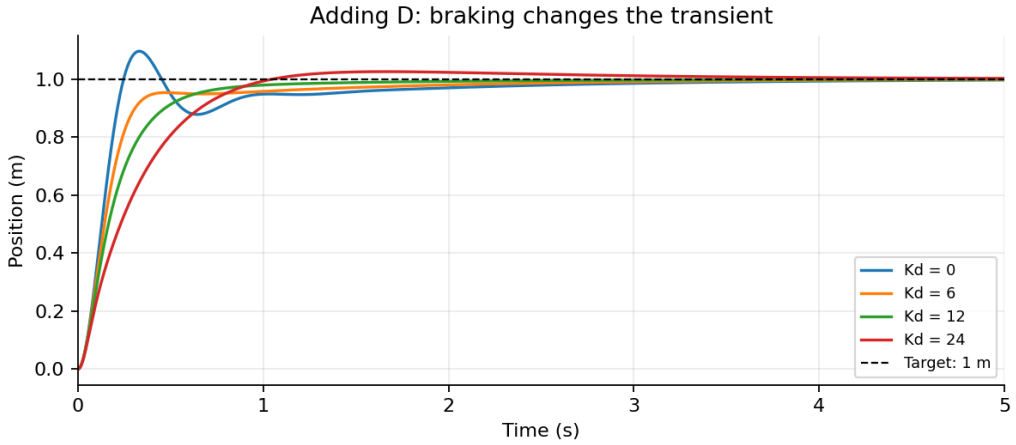


Figure 5. Increasing Kd changes the braking. The chosen Kd = 12 removes the PI overshoot in this test; excessive braking can make the approach slower.

With Ki = 80, the measured overshoot changes from 9.56% at Kd = 0 to 0.00% at Kd = 12. These plots use a 0.02 s derivative filter and no force limit, to separate the effects of the gains.

6. All together

P, PI, PD and PID are compared with the same plant, target and time step. P and PD share Kp = 100. PI and PID add Ki = 80; PD and PID use Kd = 12. The derivative filter time is 0.02 s.

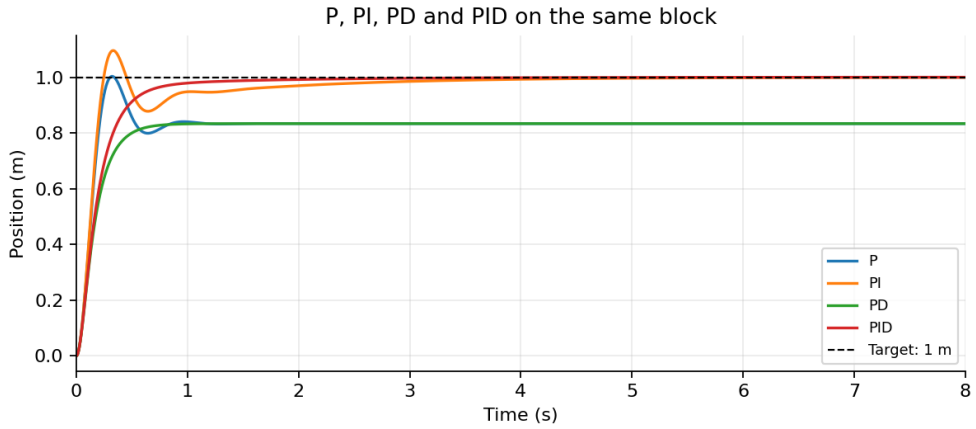


Figure 6. PI removes the offset but overshoots. PD adds damping but retains the offset. PID combines the holding force and braking.

Type	Rise (s)	Overshoot (%)	Settling (s)	Error at 12 s (m)
P	0.183	0.305	Not reached	0.166667
PI	0.164	9.562	2.602	0.000027
PD	Not reached	0.000	Not reached	0.166667
PID	0.420	0.000	1.038	0.000004

Why I kept Kd = 12

I also compared three PID settings with Kp = 100 and Ki = 80. All use dt = 0.002 s, tau = 0.02 s, no noise and no force limit, just like the comparison above.

Kp, Ki, Kd	Rise (s)	Overshoot (%)	Settling (s)
100, 80, 6	0.259	0.00	2.026
100, 80, 12	0.420	0.00	1.038
100, 80, 24	0.608	2.52	2.216

I kept **(100, 80, 12)**: it settles fastest among these three, with no measured overshoot. Kd = 6 rises faster but leaves a slower final approach. At Kd = 24, stronger braking gives I more time to build, so more D does not guarantee less overshoot. This is a small comparison, not a search for the best possible gains.

Gain units are N/m, N/(m s) and N s/m. These runs can exceed 30 N. Section 9 repeats the chosen setting with the actual force limit and noise; it takes longer to settle.

7. Practical problems

(a) Force limit and integral windup

When the actuator is already at 30 N, a larger command cannot move the block any faster. An unrestricted integral still adds error during that wait. By the time the block arrives, too much positive integral force remains. The block overshoots while that stored value unwinds.

For this demonstration I use (Kp, Ki, Kd) = (100, 200, 8), tau = 0.02 s and a symmetric +/-30 N limit. Both runs have the same gains; only conditional integration changes.

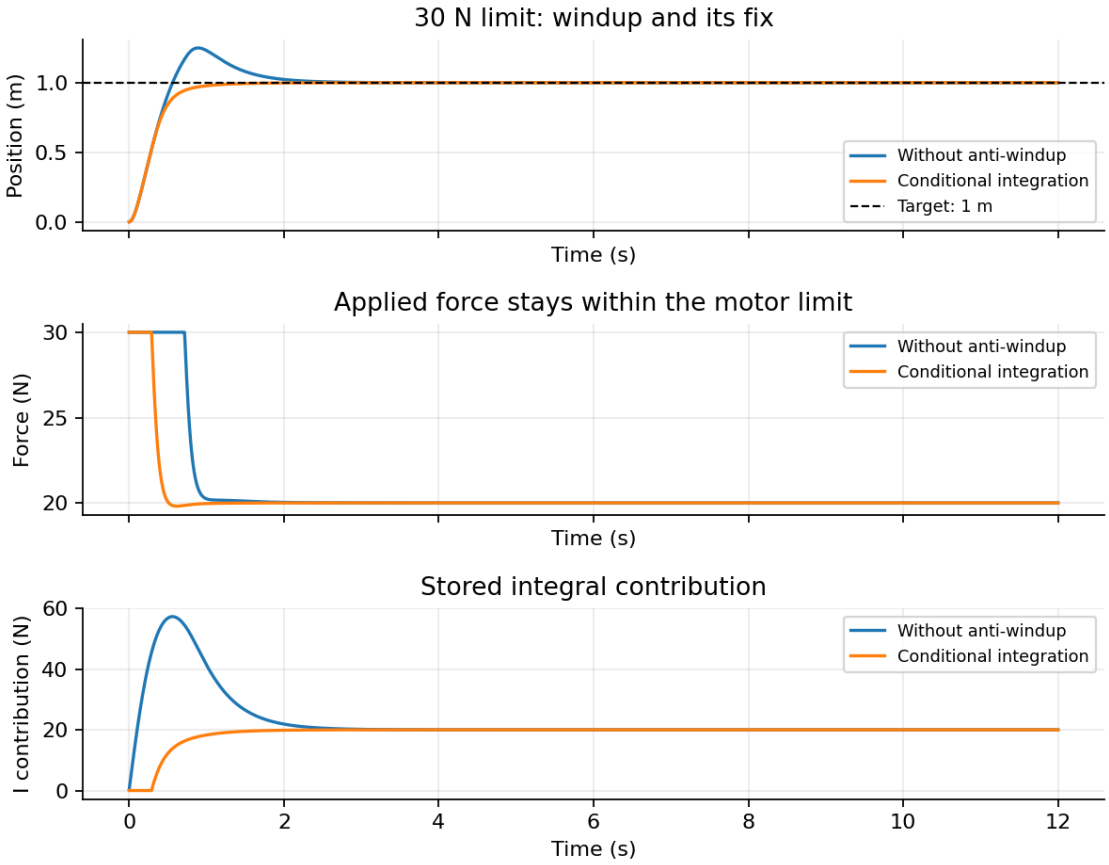


Figure 7. The output is clipped in both cases. Anti-windup stops the integral from storing an unnecessary push while saturation is active.

Overshoot falls from 24.98% to 0.00%. Settling time changes from 2.062 s to 1.034 s. Anti-windup cannot create extra actuator force or make an unreachable target reachable.

7. Noise and target changes

(b) A noisy sensor makes D jumpy

The sensor gets independent uniform noise between -0.001 and +0.001 m at every sample. Dividing small differences by dt = 0.002 s creates a large apparent velocity. The derivative filter smooths that velocity at the cost of a small delay.

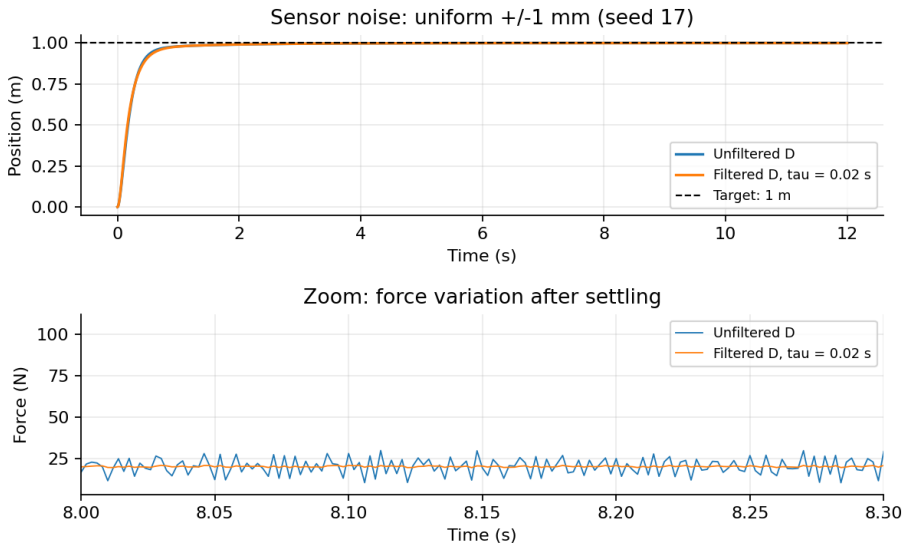


Figure 8. Both runs use identical noise samples. Filtering reduces the rapid force variation even though the position curves look similar.

From 8 to 12 s, force standard deviation, a measure of its spread around the mean, drops from 5.004 N to 0.380 N with tau = 0.02 s. The force is not limited in these runs, so clipping cannot hide the noise.

(c) Derivative kick

At t = 1 s the target jumps from 0 to 1 m. Derivative of error sees a change of 1/0.002 = 500 m/s, so Kd = 12 creates a 6000 N derivative term. Differentiating the measurement avoids that target term. P still changes immediately.

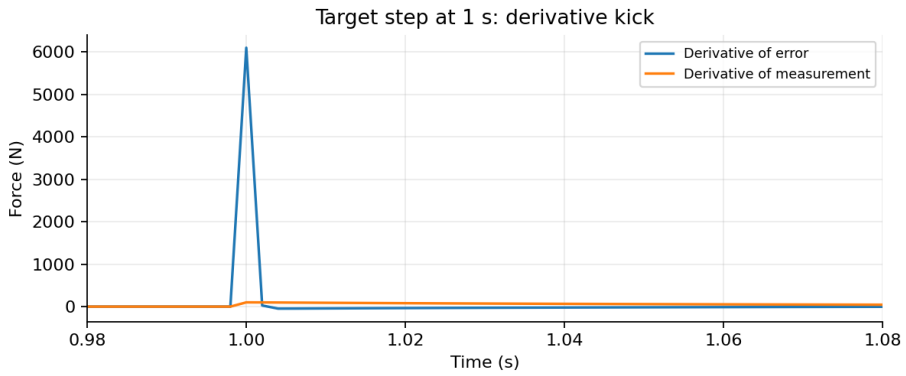


Figure 9. With no filter or saturation, peak total force is 6100.16 N for derivative of error and 100.16 N for derivative of measurement.

7. Slow controller updates

(d) Sampling time is part of the controller

A digital controller only sees the state at update times. Between updates the last force is held constant. If updates are slow, the block can move past where that force was sensible before the next correction arrives.

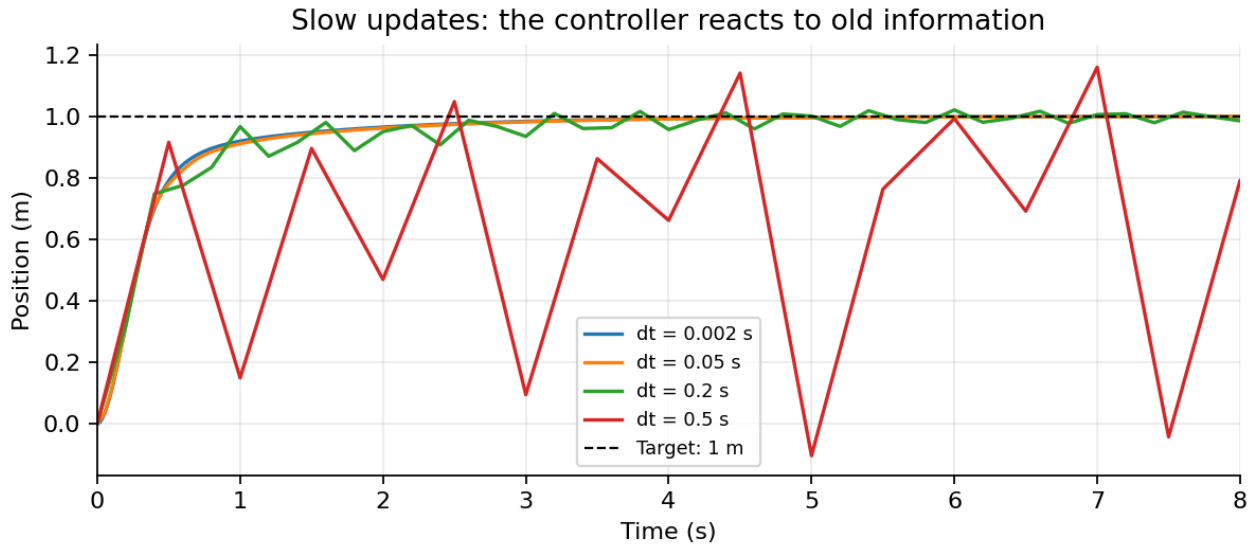


Figure 10. The gains and +/-30 N limit are fixed. Only the controller update interval changes; plant integration remains at 0.002 s or smaller.

Update interval	Overshoot (%)	Settling (s)	Error at 12 s (m)
dt = 0.002 s	0.00	2.724	0.0000
dt = 0.05 s	0.00	2.850	0.0000
dt = 0.2 s	2.10	7.600	-0.0042
dt = 0.5 s	17.10	Not reached	-0.1540

The 0.5 s controller does not settle during the 12 s run. Its last error is about -0.154 m. Returning to 0.002 s updates allows the block to settle again with these gains. The 0.2 s case is also much slower to settle than the 0.002 s case.

The plant uses the Runge-Kutta method of order four to estimate motion using four slope evaluations within each small time step. The force stays fixed until the next controller update. Keeping the plant steps small makes the graph a test of slow feedback rather than inaccurate motion calculations. A real controller should use actual elapsed time and be retuned if its update rate changes.

8. Choosing the gains

How I would tune the gains by hand

1. Check signs and units. Apply a small positive force and verify that position increases. Confirm that the target can be held within the actuator limit.
2. Start with Ki = Kd = 0. Increase Kp gradually until the response is quick enough, then reduce it if the motion rings too much.
3. Add a small Ki to remove the remaining error. Increase it slowly; back off if overshoot or slow oscillation grows.
4. Add Kd to damp the motion. Take the derivative of the measurement and filter it before testing noisy feedback.
5. Enable force limits and anti-windup. Test target changes, noise, different loads and a slower update interval. Record the same four measures after each change.

Ziegler-Nichols on a second plant

The second plant is written $G(s) = 1/(s+1)^3$. G describes how the plant responds at different frequencies using the variable s. The simulation models it as three stages in a row. Each stage takes about 1 s to cover 63% of a sudden change in its input. This lag makes the plant easier to set oscillating.

The code uses z1' = u - z1, z2' = z1 - z2, z3' = z2 - z3, y = z3. z1, z2 and z3 are the stage outputs; a prime means rate of change per second. All input and output values here are dimensionless. Use P only and raise Kp until the swings stop shrinking. That gain is Ku; time between neighbouring peaks is Tu.

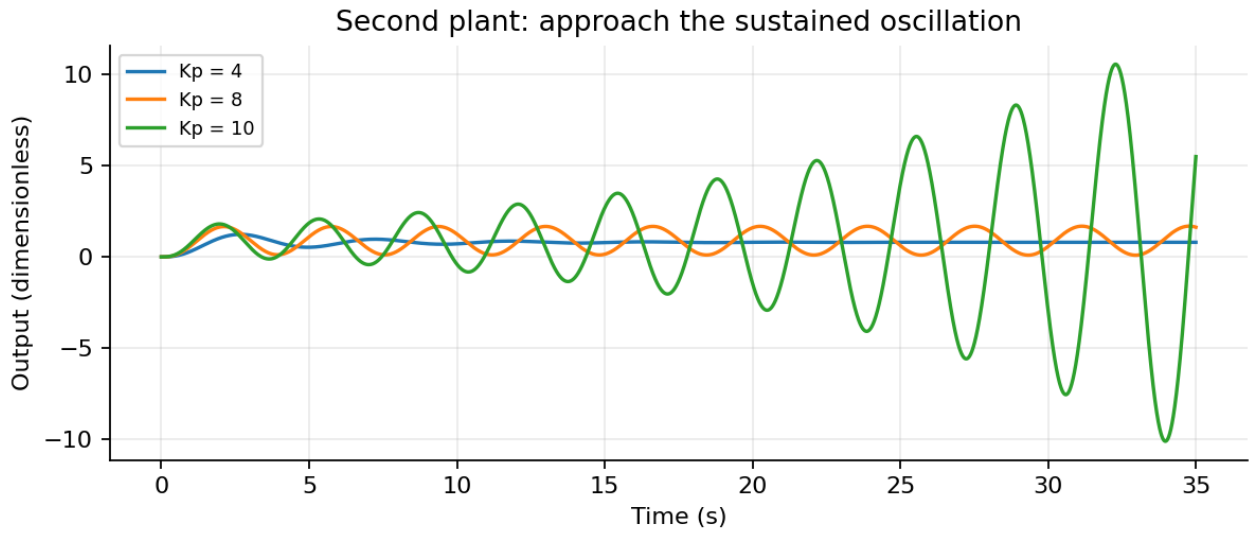


Figure 11. Kp = 4 gives decaying oscillations; Kp = 8 is close to sustained oscillation; Kp = 10 gives growing oscillations.

```
Ku = 8
Tu from simulated peak spacing = 3.6290 s
Theoretical Tu = 3.6276 s
```

I averaged the time between peaks after 10 s. Updating the controller at discrete times gives a slightly different period from theory. At Kp = 8 the swings stay roughly the same size, while at Kp = 10 they keep growing. I did this test in simulation because deliberate oscillation can be unsafe on hardware.

8. Recipe and adjustment

The Ziegler-Nichols recipe uses Ku and Tu to give starting values for Kp, Ki and Kd. Use Tu in seconds.

Controller	Kp	Ki	Kd
P	0.5 Ku	0	0
PI	0.45 Ku	0.54 Ku / Tu	0
PID	0.60 Ku	1.20 Ku / Tu	0.075 Ku Tu

```
Using Ku = 8 and theoretical Tu = 3.6276 s:
Kp = 0.60 x 8 = 4.8
Ki = 1.20 x 8 / 3.6276 = 2.6464 1/s
Kd = 0.075 x 8 x 3.6276 = 2.1766 s
```

I take the derivative of the measurement here. I leave the filter off because there is no sensor noise. The textbook recipe is a starting point. Its aggressive integral action gives a fast response but substantial overshoot on this plant. Reducing Kp and Ki gives the adjusted gains (3, 1, 2).

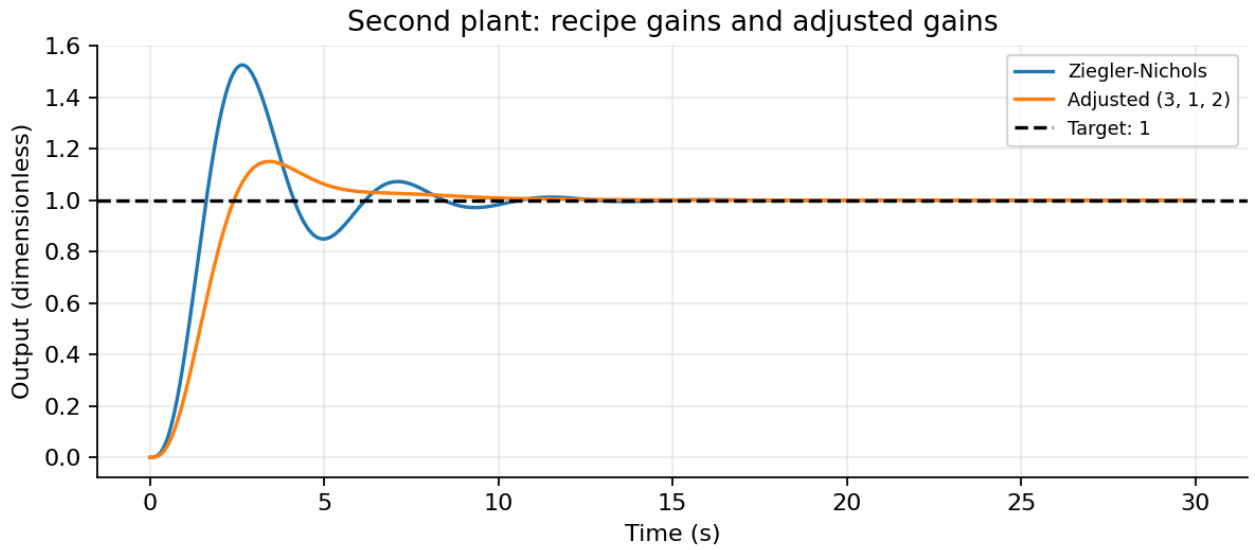


Figure 12. Adjusting the recipe reduces overshoot, with a slower rise. Neither response has an actuator limit or sensor noise.

Gains	Rise (s)	Overshoot (%)	Settling (s)
Ziegler-Nichols	0.949	52.58	9.908
Adjusted (3, 1, 2)	1.471	15.06	8.202

Overshoot drops from 52.58% to 15.06%, while rise time increases from 0.949 s to 1.471 s. I would reduce Ki further or try setpoint weighting if a lower overshoot were required. The block responds differently and uses different gain units, so I tuned it separately.

9. The code

pid.py contains a reusable PID class. It has update(target, measured, dt) and reset() methods. It keeps the integral contribution, previous derivative input and filtered derivative between updates. The controller does not know the plant equation.

simulation.py integrates either plant with RK4, holds the control output between updates, and calculates response measures. **run.py** creates all the experiments and graphs, saves the measurements, and can rebuild this document through **report.py**.

```
controller = PID(100, 80, 12, limits=(-30, 30),
tau=0.02)
run = simulate(controller, noise=0.001)
```

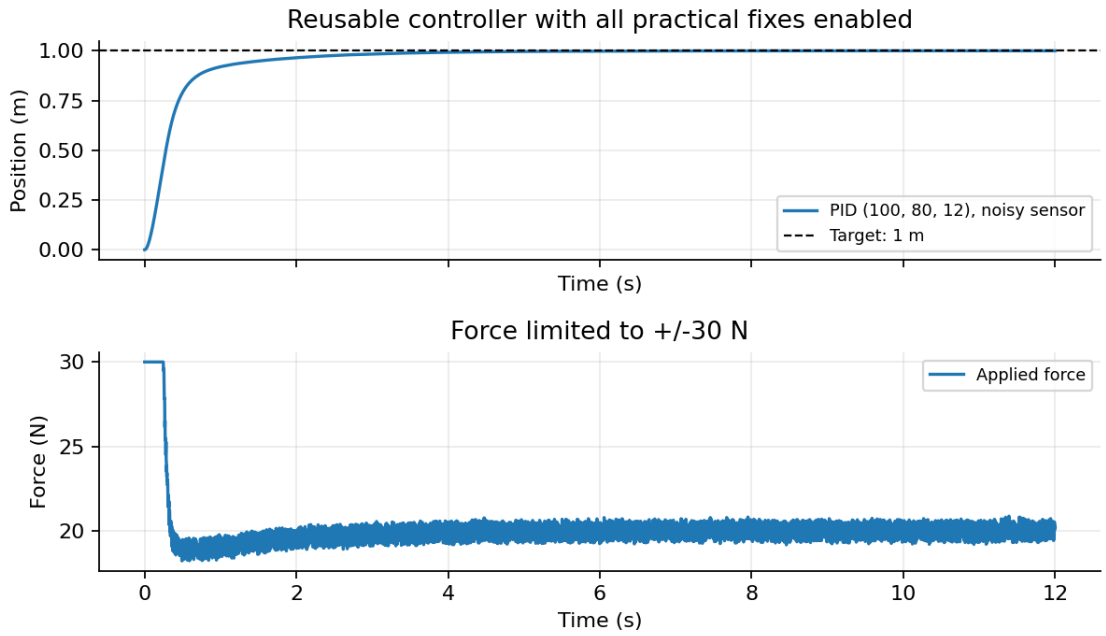


Figure 13. The chosen controller on the block with +/-30 N force, +/-1 mm sensor noise, a filtered derivative of the measurement and anti-windup.

The practical run rises in 0.729 s, overshoots by 0.011%, settles in 2.710 s, and ends with error 0.0000068 m. The actual position is used for these measures; the controller sees the noisy position. Force is always between -30 and +30 N.

Run **python run.py --report** to recreate every graph and the PDF. Results use a fixed random seed of 17. The CSV contains time, target, actual position, force and integral contribution. The README includes environment setup and commands for the checks.

10. Real robots & myths

Application	What PID does	What needs care
DC motor speed	Encoder speed is compared with the requested rpm; the output changes motor voltage or duty cycle.	Load torque, voltage limits and measurement noise at low speeds.
Drone attitude	An angle loop requests angular rate; inner rate loops use gyro feedback to request motor torque.	Fast updates, vibration and coupled axes; one PID is not the whole flight controller.
Robot arm joint	Measured joint angle is compared with the desired angle; output requests motor torque.	Gravity varies with posture; feedforward and joint limits help.
Robot that follows a line	A sensor estimates lateral line error; the correction changes the difference between wheel speeds.	Detecting when the line is lost, sharp corners and limited wheel speed.

Common wrong beliefs

'Increasing Kp always helps.' It reduces the block's offset but can increase oscillation and saturation.
'D removes error at rest.' D is zero at rest; the PD block still has a 0.1667 m offset.
'I is always harmless.' I can wind up against a force limit; this test gives 24.98% overshoot without protection.
'One set of gains works everywhere.' Plant mass, damping, sensor quality and update rate affect suitable gains.
'PID needs an exact plant model.' It can be tuned from measured responses, but a model helps explain limitations and test changes before hardware experiments.

References

University of Michigan, Control Tutorials: Introduction to PID Controller Design. Used for the model of the block, spring and damper and the interpretation of the three terms.

University of Michigan, Ziegler-Nichols PID tuning table. Used for the tuning factors. The Python code in the repository generates all the plots.

Tool use disclosure

I used AI at the start to evaluate my ideas and explore different approaches. It helped me when I got stuck on parts of the code and helped me present the report, graphs, and repository more clearly. I also used it to review my work and check that I had covered the assignment requirements.